

Loan Approval Prediction System

A Machine-Learning Approach to Credit Decisioning Project Report

Author: Sanchit Krushna Thakare | B.Tech, Electronics & Communication Engineering, VNIT Nagpur

Tools: Python, pandas, scikit-learn, Matplotlib, Seaborn | **Date:** October 2026

0.929 F1 score	0.881 Precision	0.983 Recall	0.975 ROC-AUC
--------------------------	---------------------------	------------------------	-------------------------

Hold-out test set (190 applications), Random Forest at decision threshold 0.35.

Executive Summary

This project builds a classification system that predicts whether a loan application should be approved or rejected. Using 950 labelled applications, five models were cleaned, engineered, tuned and compared. A tuned **Random Forest** was selected and evaluated once on a hold-out test set, where it reached **precision 0.88, recall 0.98 and F1 0.93**. It wrongly approved 8 of 130 applications that should have been rejected and wrongly rejected 1 of 60 that should have been approved.

Because approving a risky borrower and rejecting a good one are both costly, the models are judged on **precision, recall, F1, the confusion matrix and ROC/PR curves** rather than accuracy alone.

Key findings

- **Credit score and DTI ratio drive the decision.** In this data nobody with a credit score of 650 or below, or a DTI ratio above 0.40, was approved. They are by far the two most important features, and SHAP analysis attributes about 82% of the model's decisions to them.
- **A simple, readable policy comes close.** A depth-3 decision tree (three readable rules) reaches F1 0.91 (precision 0.83, recall 1.00). The tuned Random Forest improves precision to 0.88, i.e. fewer wrongly approved loans (8 vs 12).
- **Tree-based models clearly beat linear and kernel models** (cross-validated F1 about 0.93 vs 0.79-0.81), because the data contains sharp cut-offs that a smooth linear boundary cannot represent well.
- **Gender was excluded from the model** and audited afterwards. Recall was similar across groups; precision was somewhat lower for male applicants (0.84 vs 0.92).
- **Caveat:** the dataset appears synthetic (unusually clean cut-offs), so these scores should not be read as real-world performance.

1. Introduction and Objective

Lending decisions must be fast, consistent and defensible. Manual review is slow and varies between officers, while a purely rule-based filter may miss useful patterns. This project asks: *can a machine-learning model reproduce good credit decisions from applicant data, and which approach is best?*

Objectives

- Explore the data and understand what separates approved from rejected applications.
- Engineer features that reflect real banking logic (loan-to-income, EMI-to-income, collateral coverage).
- Compare several classifiers with proper tuning, handling class imbalance and avoiding data leakage.
- Evaluate with precision, recall, F1, confusion matrix and ROC/PR curves, and choose a decision threshold deliberately.
- Explain the model and check it for unequal treatment across gender.

2. Dataset

The dataset contains 1,000 loan applications with 19 attributes and a Yes/No approval label. 50 rows had no label and were removed, leaving **950 labelled applications, of which 298 (31.4%) were approved**. Each feature column also had about 5% missing values, which are imputed inside the modelling pipeline.

Group	Features
Applicant & household	Applicant_Income, Coapplicant_Income, Age, Dependents, Marital_Status, Education_Level, Gender
Employment	Employment_Status, Employer_Category
Credit profile	Credit_Score, Existing_Loans, DTI_Ratio (debt-to-income), Savings
Loan request	Loan_Amount, Loan_Term, Loan_Purpose, Collateral_Value, Property_Area
Target	Loan_Approved (Yes / No)

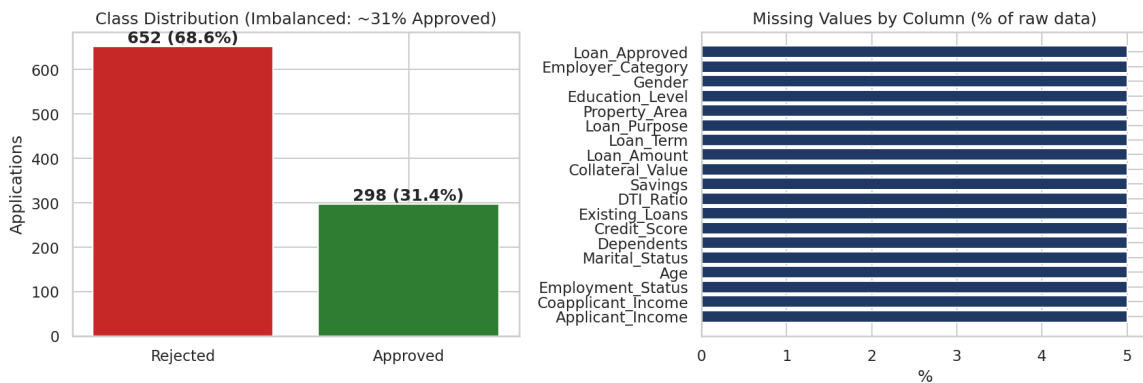


Figure 1. Class distribution (left) and share of missing values per column (right). Classes are imbalanced, so class weighting and F1 are used.

3. Exploratory Data Analysis

The charts below summarise the main patterns. The strongest correlations with approval are credit score (+0.49) and DTI ratio (-0.47); loan amount (-0.13) and applicant income (+0.13) are much weaker.

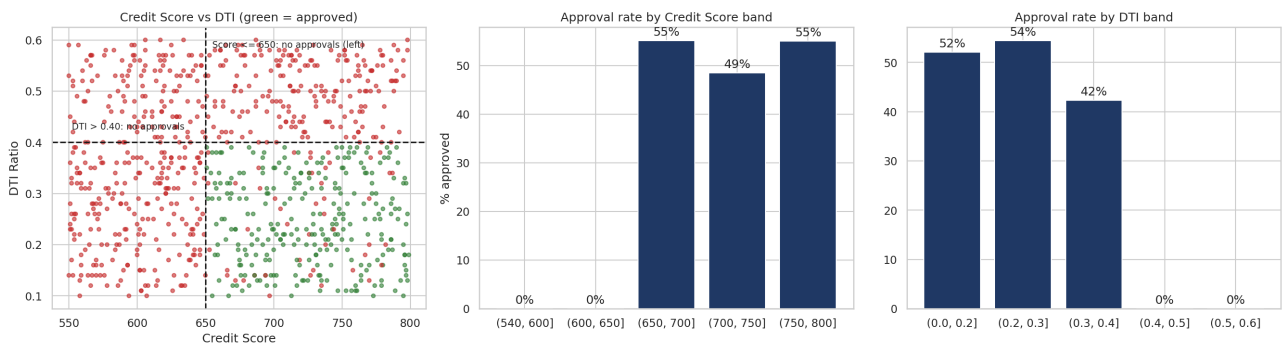


Figure 2. Credit score and DTI define clear approval zones. No application with a score of 650 or below, or a DTI above 0.40, was approved. Inside the eligible zone, approval is roughly 50%, so other factors decide those cases.

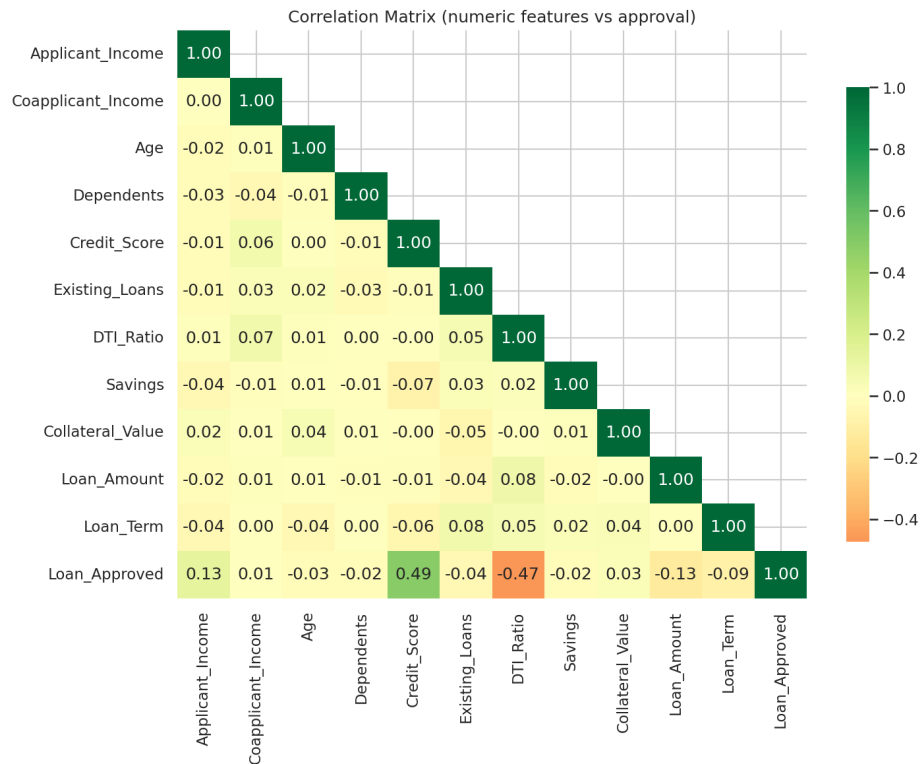


Figure 3. Correlation matrix of numeric features and the approval outcome.

Feature Distributions: Approved vs Rejected

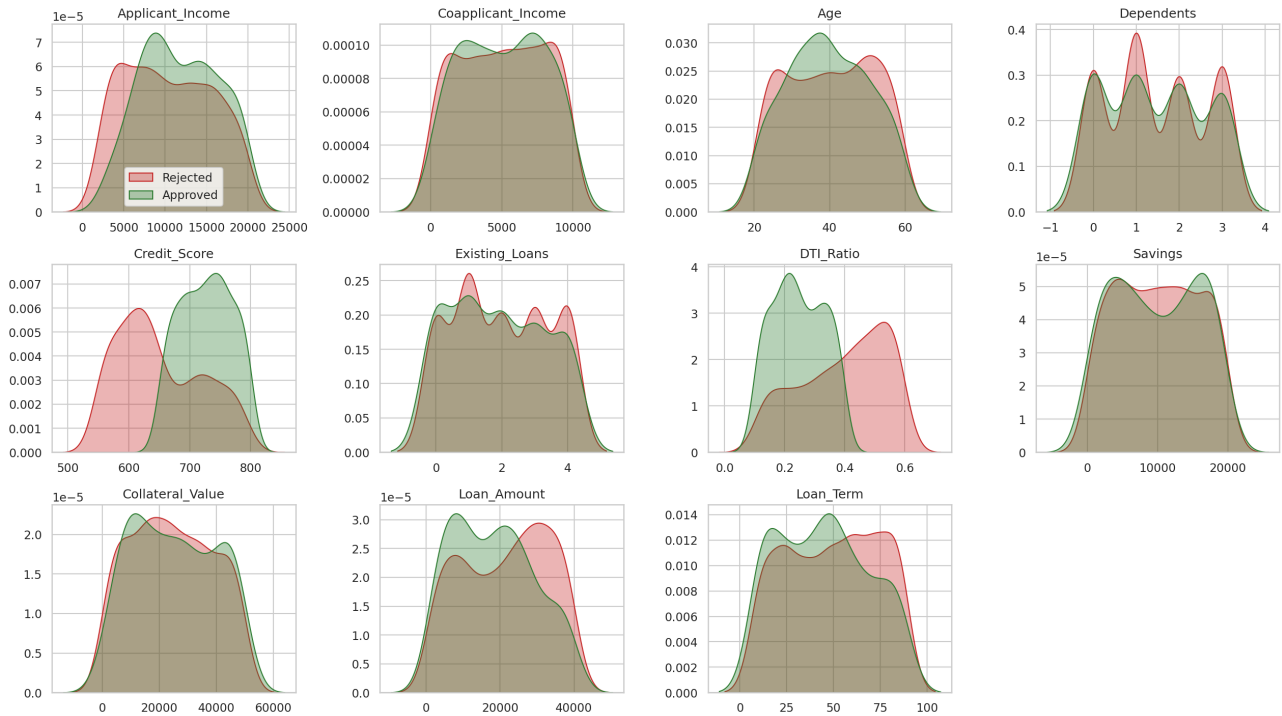


Figure 4. Distributions of each numeric feature for approved and rejected applications.

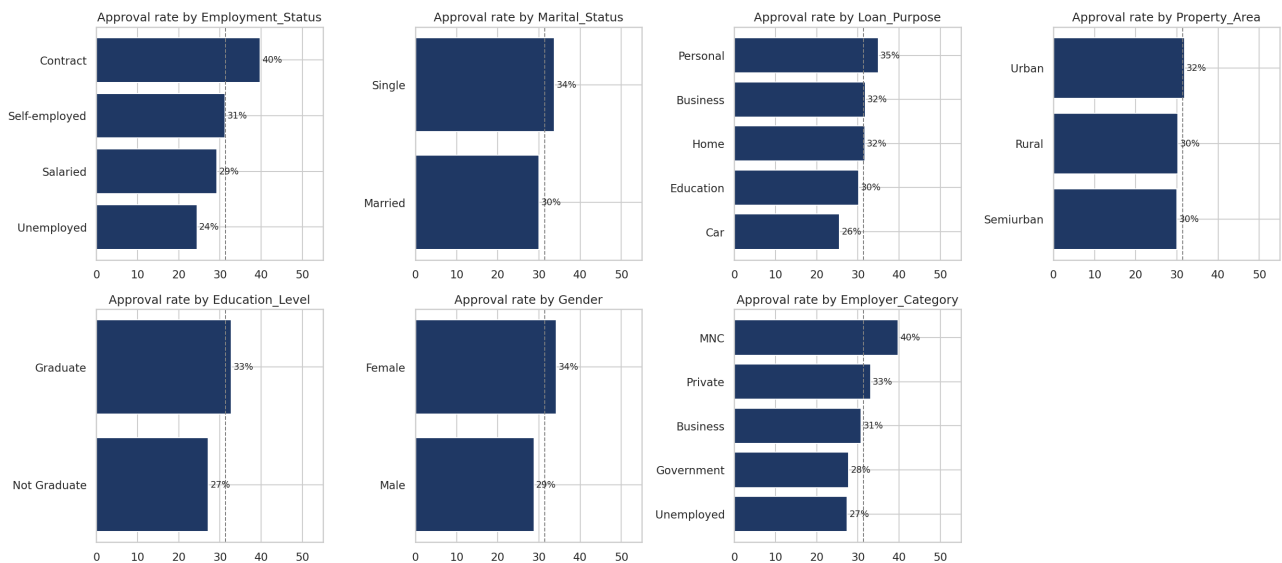


Figure 5. Approval rate by category (dashed line = overall 31.4%). Differences are modest and group sizes are small, so they should not be over-interpreted.

4. Methodology

4.1 Pipeline overview

Cleaning and imputation → feature engineering → stratified train/test split → model comparison with tuning → threshold selection → single test evaluation → explainability and fairness audit.

4.2 Data preparation

- Removed 50 unlabelled rows and the ID column.
- Numeric missing values were filled with the median and categorical ones with the most frequent value. Imputers are fitted inside the pipeline on training folds only, so no information leaks from validation or

test data.

- Numeric features were standardised and categorical features one-hot encoded.
- **Gender** was excluded as a protected attribute and used only afterwards for the fairness audit.

4.3 Feature engineering (banking ratios)

Feature	Definition	Banking meaning
Total_Income	Applicant + coapplicant income	Household repayment capacity
Loan_to_Income	Loan amount / total income	Size of the loan relative to earnings
Approx_EMI	Loan amount / loan term	Rough monthly instalment (interest not modelled)
EMI_to_Income	Approx EMI / total income	Share of income consumed by repayments
Collateral_Coverage	Collateral value / loan amount	Security cover for the lender
Savings_to_Loan	Savings / loan amount	Financial buffer
Income_per_Dependent	Total income / (dependents + 1)	Income stretched across the household

Engineered banking ratios separate approved from rejected applicants

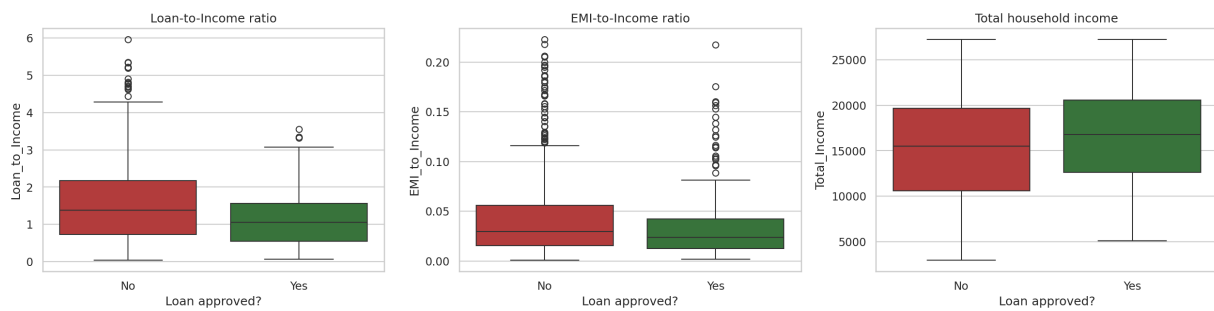


Figure 6. Engineered ratios for approved vs rejected applicants (top 2% trimmed for readability).

4.4 Train/test split and model tuning

A stratified 80/20 split gave **760 training and 190 test applications** (test set: 60 approved, 130 rejected). The test set was not used for any decision until the final evaluation. Five classifiers were compared: Logistic Regression, Decision Tree, Random Forest, Gradient Boosting (histogram-based) and an RBF Support Vector Machine. Each was tuned with randomised search (25 candidate settings, 5-fold stratified cross-validation, F1 as the objective). Class weighting handled the 69/31 imbalance.

4.5 Decision threshold

Models output an approval probability. The cut-off was chosen by maximising F1 on **out-of-fold predictions from the training data only**, giving a threshold of **0.35**. A bank could move this threshold up for stricter lending or down for more inclusive lending, depending on the cost of defaults versus lost customers.

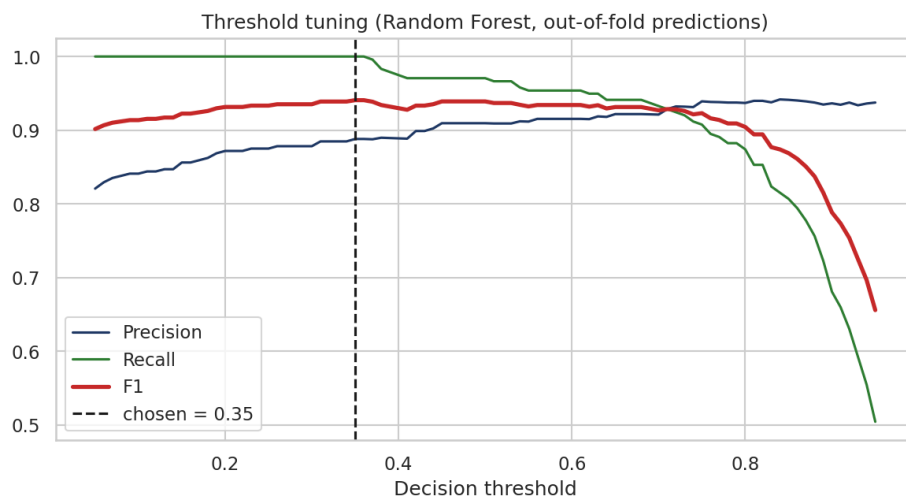


Figure 7. Precision, recall and F1 as the decision threshold varies (out-of-fold training predictions).

5. Results

5.1 Model comparison: cross-validation (training data)

Model	Precision	Recall	F1	ROC-AUC	F1 std
Random Forest	0.906	0.971	0.937	0.982	0.020
Gradient Boosting	0.897	0.975	0.934	0.980	0.026
Decision Tree	0.882	0.962	0.920	0.961	0.030
SVM (RBF)	0.796	0.823	0.808	0.939	0.026
Logistic Regression	0.733	0.861	0.791	0.924	0.009

Random Forest and Gradient Boosting are statistically close; Random Forest had the highest cross-validated F1 and was selected. Linear and kernel models trail by about 13 F1 points.

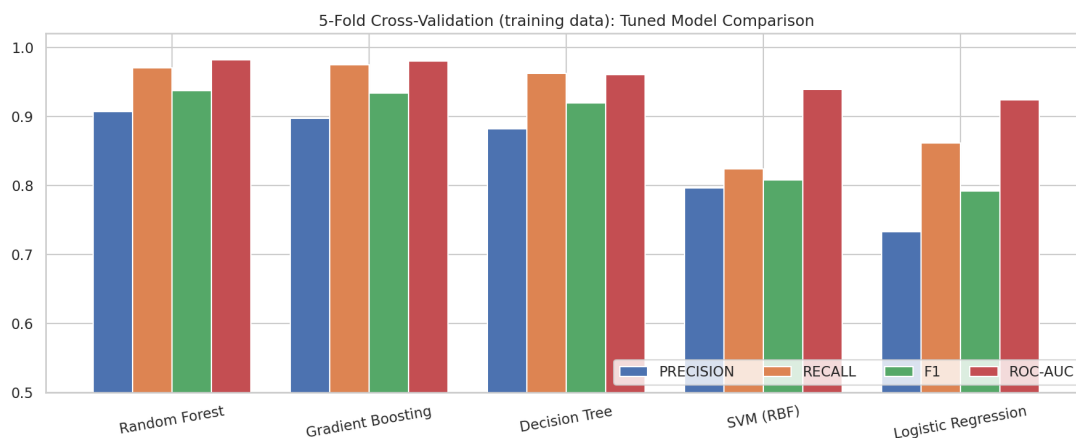


Figure 8. Cross-validated comparison of the five tuned models.

5.2 Final model on the hold-out test set

Metric	Threshold 0.35 (selected)	Threshold 0.50 (reference)
Accuracy	0.953	0.958
Precision (approved)	0.881	0.894
Recall (approved)	0.983	0.983
F1 score	0.929	0.937
ROC-AUC	0.975	0.975
PR-AUC	0.909	0.909
Matthews correlation	0.897	0.907

The two thresholds give nearly identical results, which shows the model is not sensitive to this choice.

Class	Precision	Recall	F1	Support
Rejected	0.992	0.938	0.964	130
Approved	0.881	0.983	0.929	60
Weighted average	0.957	0.953	0.953	190

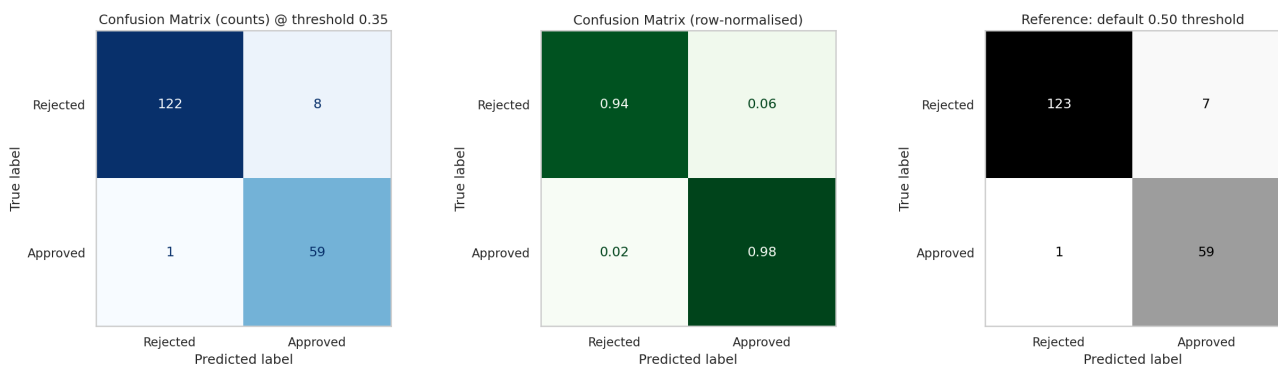


Figure 9. Confusion matrices. At threshold 0.35: 122 correct rejections, 8 wrongly approved (false positives), 1 wrongly rejected (false negative), 59 correct approvals.

Reading the confusion matrix. The model missed only 1 genuinely approvable applicant, which protects customer experience. The main error type is approving 8 applications that should have been rejected; in a real bank this is the costlier mistake and is the first thing a threshold review would target.

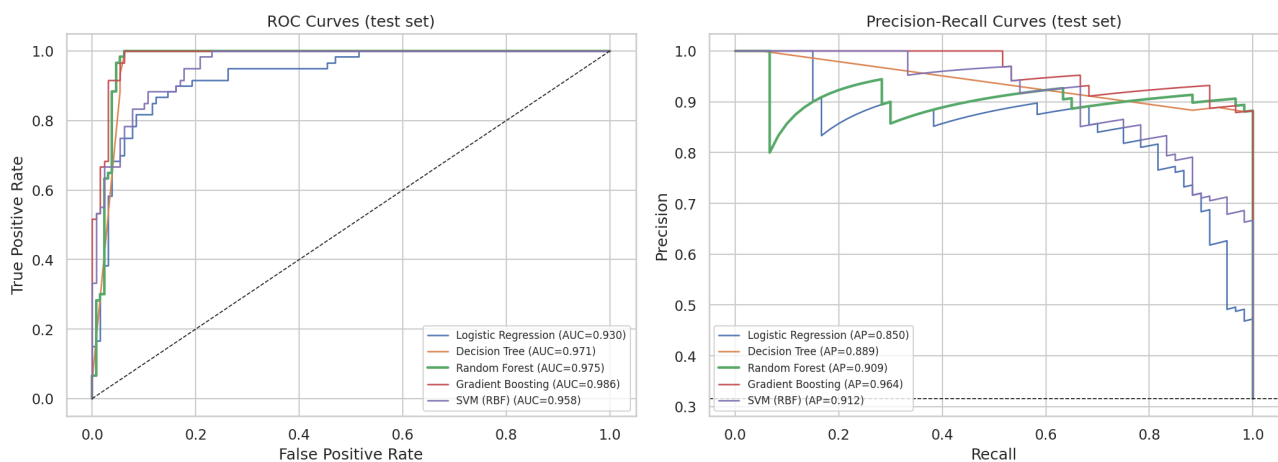


Figure 10. ROC and precision-recall curves for all tuned models on the test set.

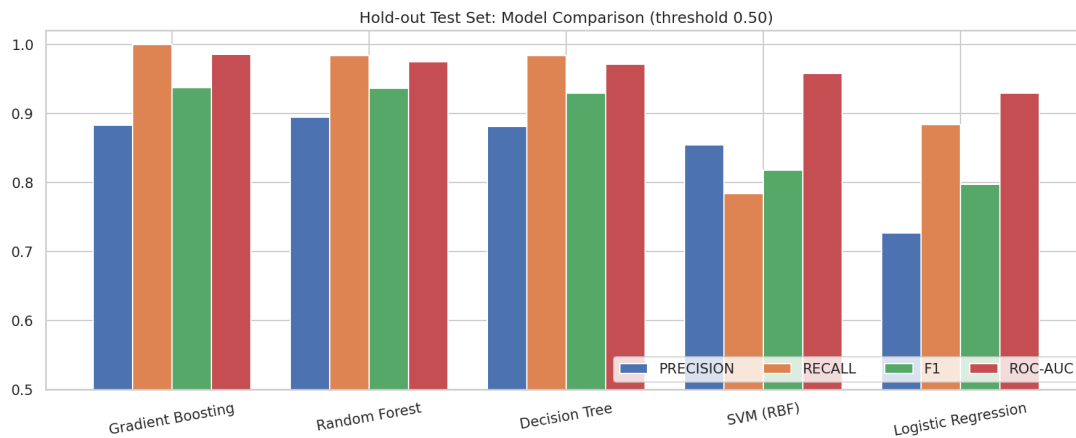


Figure 11. Test-set comparison of all five models at threshold 0.50.

Model (test, threshold 0.50)	Accuracy	Precision	Recall	F1	ROC-AUC
Gradient Boosting	0.958	0.882	1.000	0.938	0.986
Random Forest	0.958	0.894	0.983	0.936	0.975
Decision Tree	0.953	0.881	0.983	0.929	0.971
SVM (RBF)	0.889	0.855	0.783	0.817	0.958
Logistic Regression	0.858	0.726	0.883	0.797	0.930

Gradient Boosting scored marginally higher on the test set (F1 0.938), but with 190 test rows the difference from Random Forest (0.937) is within noise. Selection was made on cross-validation, not on test results.

6. Feature Engineering Ablation

Feature set (Random Forest, 5-fold CV)	Precision	Recall	F1	ROC-AUC
Raw features only	0.913	0.970	0.941	0.984
Raw + engineered banking ratios	0.910	0.971	0.939	0.983

For the Random Forest the engineered ratios did **not** improve performance (F1 0.941 vs 0.939, within noise). A plausible reason is that tree-based models can learn such interactions themselves. The ratios are still valuable for explainability: they let the credit-policy tree below be written in business terms. The effect on linear models was not part of this ablation.

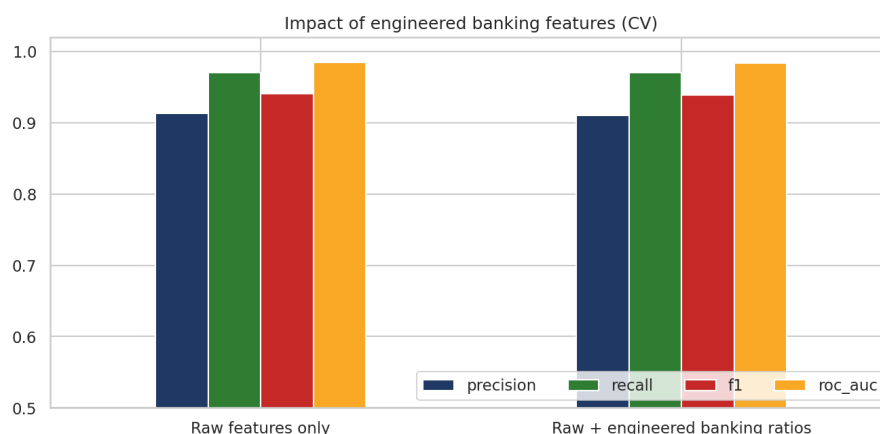


Figure 12. Cross-validated effect of the engineered features on the Random Forest.

7. Explainability

7.1 Feature importance

Permutation importance measures how much the F1 score drops when a feature is shuffled. **Credit_Score** and **DTI_Ratio** dominate (each costs the model roughly 0.3-0.4 F1 when removed), followed by Applicant_Income and Loan_to_Income. Most other features, including employment, purpose and property area, contribute almost nothing.

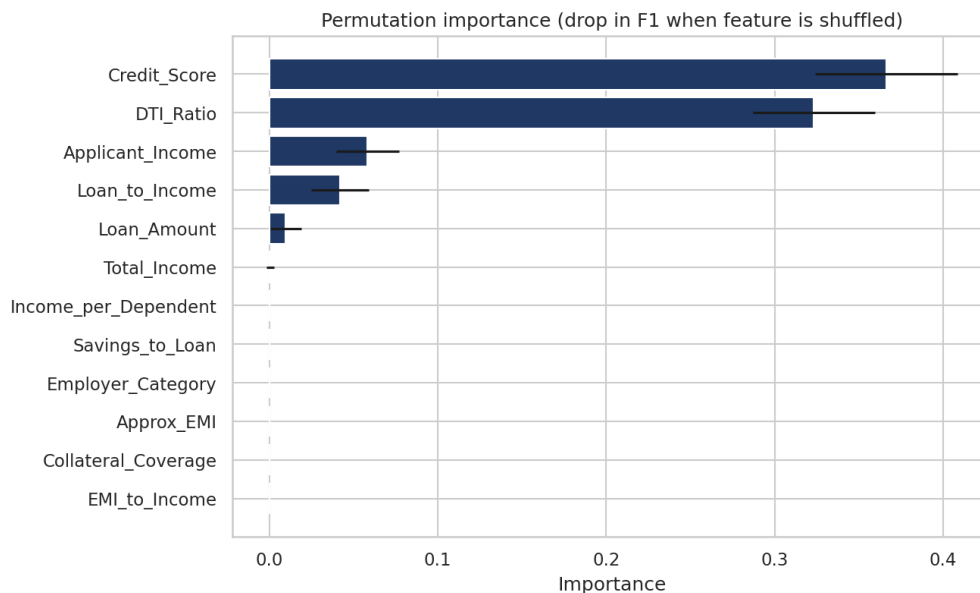


Figure 13. Permutation importance on the test set (error bars show variation over 30 shuffles).

7.2 A readable credit policy

A shallow depth-3 decision tree was fitted to translate the behaviour into plain rules:

1. If Credit_Score ≤ 650 -> REJECT
2. Else if DTI_Ratio > 0.39 -> REJECT
3. Else if Loan_to_Income > 3.57 -> REJECT
4. Otherwise -> APPROVE

On the test set this simple policy achieves **precision 0.833, recall 1.000 and F1 0.909** (12 wrongly approved, 0 wrongly rejected). The tuned Random Forest reaches F1 0.929 with 8 wrongly approved and 1 wrongly rejected. The extra accuracy of the ensemble is therefore modest. In a regulated setting where decisions must be explained to customers, the interpretable tree is a serious alternative, and the Random Forest is best viewed as a more precise second opinion.

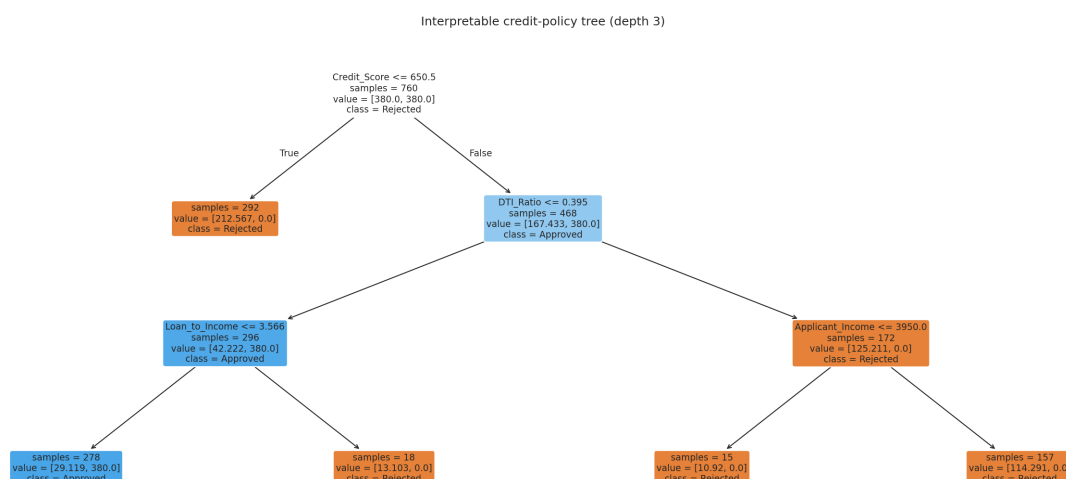


Figure 14. Depth-3 decision tree used as a readable credit policy.

7.3 SHAP analysis: which features contribute most?

SHAP (SHapley Additive exPlanations) splits every individual prediction into contributions from each feature. For this model, a SHAP value is the number of **probability points a feature adds (+) or removes (-)** for one applicant, relative to the average prediction. The values were computed with TreeExplainer on the 190 test applications, and one-hot encoded columns were merged back into their original feature (SHAP values are additive, so this is valid). The baseline is 0.50 rather than the 31% approval rate because the Random Forest was trained with class weighting, which also explains why the tuned decision threshold is 0.35 and not 0.50.

Rank	Feature	Mean SHAP	Share of total	Direction of effect
1	Credit_Score	0.272	43.3%	Higher value, more likely approved
2	DTI_Ratio	0.242	38.6%	Higher value, less likely approved
3	Applicant_Income	0.050	8.0%	Higher value, more likely approved
4	Loan_to_Income	0.030	4.8%	Higher value, less likely approved
-	All 20 other features	0.033	5.1%	Individually negligible

Credit score and DTI ratio together account for about 82% of the model's total attribution. Applicant income and loan-to-income add a small amount, and the remaining features contribute almost nothing. This matches the permutation-importance ranking in section 7.1 and the first two splits of the policy tree in section 7.2.

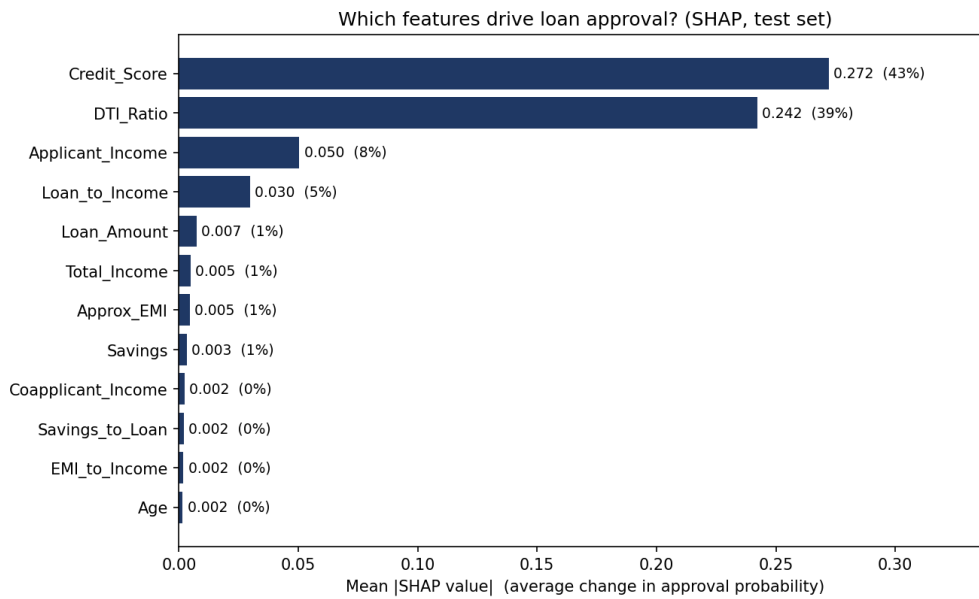


Figure 15. Mean absolute SHAP value per feature (test set). Bars show the average change in approval probability.

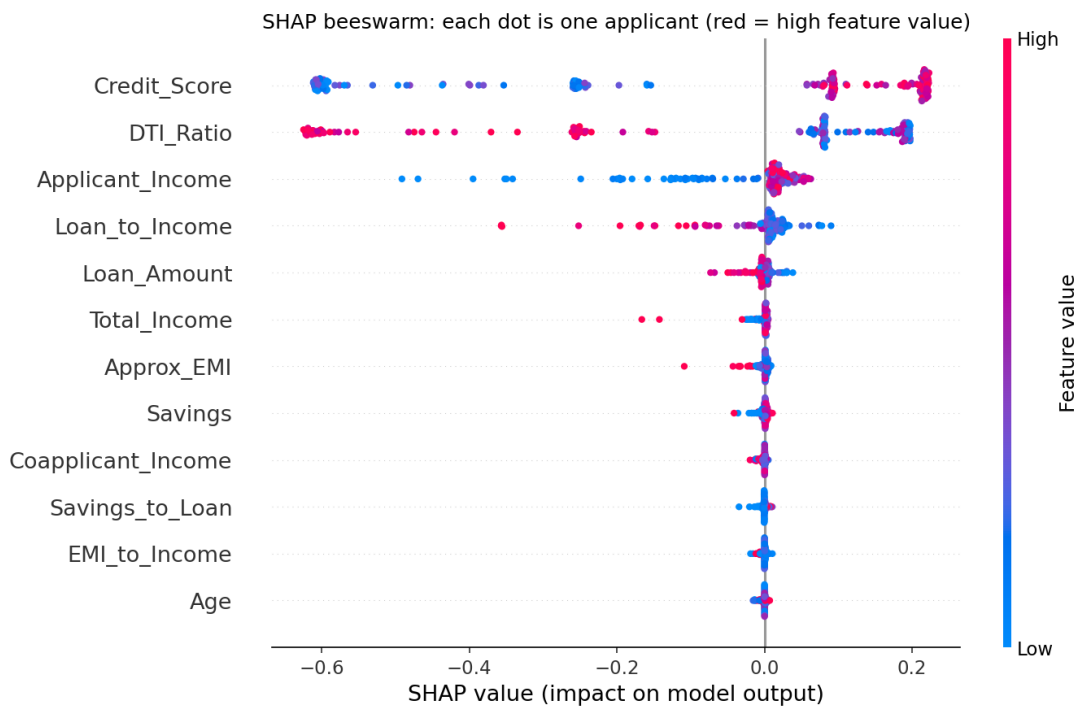


Figure 16. SHAP beeswarm. Each dot is one applicant; red means a high feature value, blue a low one. Dots to the left reduce the approval probability, dots to the right raise it.

Reading the beeswarm. Low credit scores (blue) sit far to the left and high credit scores (red) to the right, so a strong score raises the chance of approval. For DTI the pattern is reversed: high DTI values (red) push the decision strongly towards rejection. Loan-to-Income behaves like DTI, with large ratios pulling the probability down.

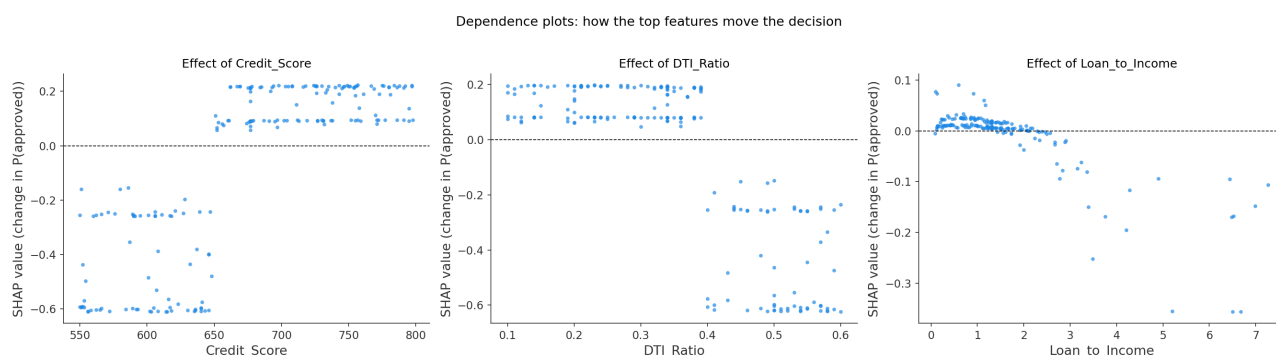


Figure 17. Dependence plots for credit score, DTI ratio and loan-to-income. Each shows how the feature's value changes its contribution to the decision.

The effects are thresholds, not smooth trends. The credit-score contribution jumps from clearly negative to positive at about **650**, and the DTI contribution falls sharply once DTI exceeds about **0.40**. Loan-to-income is slightly positive at low values and turns negative as the loan grows beyond roughly two to three times income. This step-like behaviour is the reason tree-based models outperformed logistic regression, which assumes a smooth, linear effect.

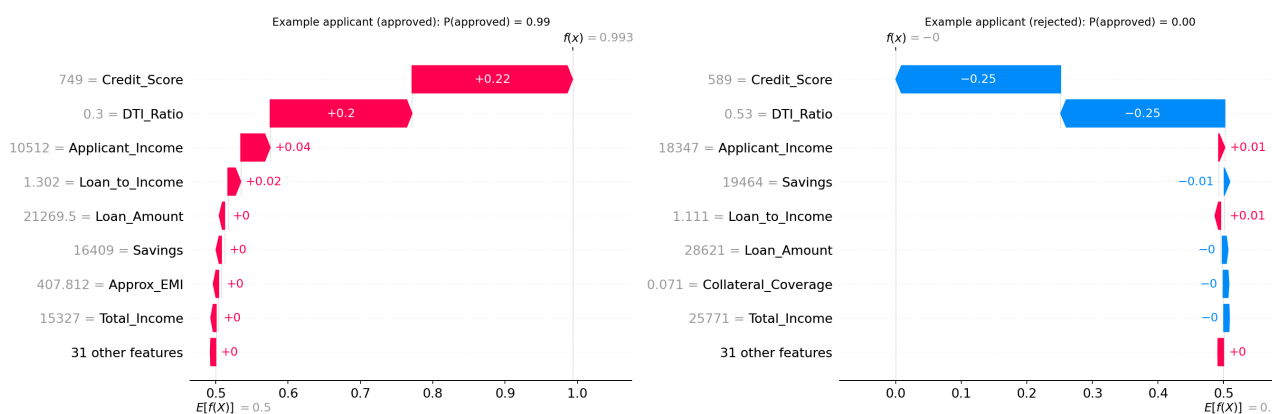


Figure 18. SHAP waterfall plots for one confidently approved applicant (left) and one confidently rejected applicant (right). Starting from the 0.50 baseline, each bar shows how a feature moves the probability.

Individual explanations. For the approved applicant (credit score 749, DTI 0.30), credit score adds about +0.22 and DTI about +0.20, taking the probability to 0.99. For the rejected applicant (credit score 589, DTI 0.53), each of these two features subtracts about 0.25, taking the probability to roughly zero. In practice such breakdowns could be turned into plain reason codes, for example "credit score below 650" or "DTI above 0.40", which helps when a decision has to be explained to a customer.

Caution. SHAP explains how the *model* behaves, not why borrowers default in the real world, and it inherits the characteristics of this dataset, which appears synthetic. The sharp thresholds seen here may be softer in real lending data.

8. Fairness Audit

Gender was not an input to the model. To check for unequal outcomes, out-of-fold predictions across all 950 labelled applications were grouped by gender:

Group	Applicants	Actual approval rate	Predicted approval rate	Recall	Precision
Female	365	34.2%	37.3%	1.000	0.919
Male	538	28.8%	33.5%	0.981	0.844
Unknown (gender missing)	47	38.3%	42.6%	1.000	0.900

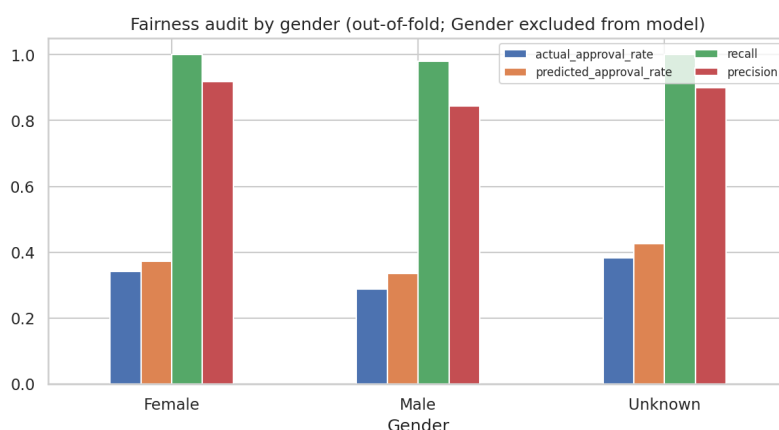


Figure 19. Outcome metrics by gender (out-of-fold predictions, gender excluded from the model).

Recall is essentially equal across groups, so genuine applicants are found at similar rates. Precision is lower for male applicants (0.84 vs 0.92), meaning more wrongly approved loans in that group. Predicted approval rates are slightly higher than actual rates in every group, consistent with the model favouring recall. **Limits of this check:** removing a protected attribute does not guarantee that other features do not act as proxies, and group sizes are small. A production system would need a fuller fairness review.

9. Limitations

- **Synthetic-looking data.** The cut-offs at credit score 650 and DTI 0.40 are unusually sharp. Real bank data is noisier, so real-world scores would be lower.
- **Small test set.** 190 applications (60 approvals) means metrics could move by a few points on a different split.
- **Mild optimism in cross-validation.** Hyper-parameters were tuned on the same training data used for cross-validation scores; the untouched test set is the cleaner estimate.
- **Simplified EMI.** The data has no interest rate, so Approx_EMI is principal divided by term.
- **Threshold chosen by F1.** A real lender would choose it using the monetary cost of defaults versus lost customers.
- **Not a production system.** It has no human review, regulatory checks or monitoring.

10. Conclusion and Future Work

A tuned Random Forest predicts loan approval with F1 0.93 (precision 0.88, recall 0.98) on unseen data, clearly outperforming logistic regression and an RBF SVM. The analysis also shows that most of the decision logic is captured by credit score, DTI ratio and loan-to-income, and that a three-rule policy performs nearly as well, which is valuable for transparency. SHAP analysis confirms that credit score and DTI ratio account for about 82% of the model's attribution and that both act as sharp thresholds, at roughly 650 and 0.40.

Possible extensions

- Test on real, noisier lending data and use nested cross-validation for a less optimistic estimate.
- Replace the F1-based threshold with a cost-sensitive threshold using estimated loss from defaults.
- Model EMI with an interest rate and add repayment-history features.
- Turn the SHAP breakdowns into customer-facing reason codes and add probability calibration.
- Extend the fairness review (more groups, intersectional checks) and add drift monitoring.

Appendix A. Reproducibility

Item	Detail
Random seed	42 (split, tuning, models)
Selected model	Random Forest: n_estimators=364, max_depth=8, max_features=0.8, min_samples_leaf=3, class_weight=balanced_subsample
Environment used	Python 3.12, scikit-learn 1.8.0, pandas 3.0.2, numpy 2.4.4, shap 0.52.0 (see requirements-tested.txt)
Run	python src/loan_approval.py (or notebooks/loan_approval_notebook.ipynb), then python src/shap_analysis.py for the SHAP figures
Note	Results can shift by a point or two on different library versions.